

Figure 4.11 Flowchart of how the Probability of Improvement score is computed. By following this procedure, we can compute how likely any candidate point is able to improve from the incumbent.

> **NOTE** **Technically, the CDF computes the area of the portion of a normal distribution to the left of a threshold, so we need to subtract from 1 the output of the CDF to get the area of the portion to the right of the threshold, which corresponds to possible improvements.**

We first generate the GP's prediction at this point:

```
with torch.no_grad():
    predictive_distribution = likelihood(model(torch.tensor([[0.]])))
    predictive_mean = predictive_distribution.mean   ❶
    predictive_sd = predictive_distribution.stddev   ❷
```

❶ Predictive mean at 0.

❷ Predictive standard deviation at 0.

Then, we initialize a one-dimensional normal distribution with the corresponding mean and standard deviation:

```
normal = torch.distributions.Normal(predictive_mean, predictive_sd)
```

This normal distribution is the one visualized in the right panel of Figure 4.9. Finally, to compute the area of the highlighted portion, we call the `cdf()` method with the incumbent as its input (which is `train_y.max()`, the maximum value of our training data) and subtract the result from 1:

```
>>> 1 - normal.cdf(train_y.max())

tensor([0.8305])
```

Here, our code shows that at point 0, we have more than 80 percent of improving from the incumbent, which agrees with the fact that a large portion of the normal distribution in Figure 4.9 is highlighted. Using the same computation, we can find out that there is 0.4948 probability of

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>